

Statistical and Machine Learning

Neural Networks for Tabular Data

Instructions

All questions should first be discussed in pairs or small groups. Afterwards, we shall discuss them together. Solutions will be distributed after each session.

Question 1

Many simple models, such as linear and logistic regression, are linear in the input $\mathbf{x}$. One way to make them more flexible is **basis function expansion**, replacing $\mathbf{x}$ with $\phi(\mathbf{x})$ before the linear step.

- Give an example of such a transform ϕ for a one-dimensional input.
- What is the main limitation of this approach?
- How do deep neural networks address this limitation?
- Write down the general formula for a deep feedforward network with L layers, expressed as a composition of functions $f_1, \dots, f_L$. Then write the formula describing how a single layer's output $\mathbf{z}_l$ is computed from the previous layer's output $\mathbf{z}_{l-1}$, in terms of weights $\mathbf{W}_l$, bias $\mathbf{b}_l$, and activation function φ_l .

Answer:

- A polynomial transform, e.g. $\phi(x) = [1, x, x^2, x^3, \dots]$.
- ϕ has to be designed by hand, which limits the kinds of functions the model can represent.
- DNNs give the feature extractor its own learnable parameters, $\phi(\mathbf{x}; \boldsymbol{\theta}_2)$, so the features are *learned from data* jointly with the rest of the model rather than fixed in advance. Stacking many such layers builds increasingly abstract learned features.
- $f(\mathbf{x}; \boldsymbol{\theta}) = f_L(f_{L-1}(\dots f_1(\mathbf{x}) \dots))$, with each layer computed recursively as $\mathbf{z}_l = \varphi_l(\mathbf{b}_l + \mathbf{W}_l \mathbf{z}_{l-1})$.

Question 2

The XOR function cannot be implemented by a single perceptron (a thresholded linear combination of the inputs).

- Why not – what property of the XOR classes is the problem?
- Briefly explain how stacking two hidden units, one computing AND and one computing OR, lets a multilayer perceptron solve XOR.
- What general lesson does this example teach about the role of hidden units in a network?

Answer:

- The two classes of XOR are not *linearly separable*: no single straight line in the (x_1, x_2) plane separates the 0s from the 1s, but a perceptron's decision boundary is always a hyperplane.
- XOR is true when exactly one of the two inputs is equal to 1. Equivalently,

$$\text{XOR}(x_1, x_2) = \text{OR}(x_1, x_2) \text{ AND NOT}(\text{AND}(x_1, x_2)).$$

A hidden layer can compute the two intermediate quantities $h_1 = \text{AND}(x_1, x_2)$, and $h_2 = \text{OR}(x_1, x_2)$. The output layer then combines them by activating when $h_2 = 1$ but $h_1 = 0$. Equivalently, it computes something like $h_2 - h_1$. This gives

x_1	x_2	h_1	h_2	$h_2 - h_1$
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Thus, the hidden layer constructs useful features that make the XOR pattern linearly separable at the output layer. This is why a multilayer perceptron can solve XOR, whereas a single linear perceptron cannot.

- c) Hidden units let a network construct new, more useful intermediate features that are not present in the raw input – this is what gives multilayer networks extra expressive power over a single layer/perceptron.

Question 3

Suppose someone proposes building a deep network using only **linear** activation functions, to keep things simple.

- a) Explain why this would be a bad idea.
- b) Write down the sigmoid activation function $\sigma(a)$ and its derivative $\sigma'(a)$. For what values of $\sigma(a)$ does this derivative become (approximately) zero, and why does this matter for training deep networks?
- c) Write down the ReLU function and its derivative. Why does this avoid the vanishing-gradient problem that affects the sigmoid?
- d) Is ReLU completely free of issues? If not, name one problem it has.

Answer:

- a) Stacking only linear layers collapses into a single linear map overall, $\mathbf{W}_L \mathbf{W}_{L-1} \cdots \mathbf{W}_1 \mathbf{x} = \mathbf{W}' \mathbf{x}$, so no matter how many layers are added the network can never represent more than a linear model: nonlinearity is essential for any extra expressive power.
- b) $\sigma(a) = \frac{1}{1 + e^{-a}}$, with derivative $\sigma'(a) = \sigma(a)(1 - \sigma(a))$. This is close to zero exactly when $\sigma(a)$ is close to 0 or 1 – i.e. when the sigmoid saturates for large $|a|$. It matters because, in the chain rule used by backpropagation, these near-zero factors get multiplied together across layers, so the gradient signal reaching earlier layers shrinks toward zero (the vanishing gradient problem).
- c) $\text{ReLU}(a) = \max(a, 0)$, with derivative $\text{ReLU}'(a) = \mathbb{I}(a > 0)$ (1 if $a > 0$, else 0). For positive inputs the derivative is exactly 1, not a fraction less than 1, so the gradient passes through unchanged rather than shrinking as it is propagated backward.
- d) It is not problem-free: the **dead ReLU** problem occurs when a unit's pre-activation becomes negative for all inputs (e.g. due to poor initialization); its output and gradient are then permanently zero, and the unit can never recover.

Question 4

Backpropagation can be seen as a special case of automatic differentiation.

- a) What quantity does backpropagation compute, and why do we need it to train a network?
- b) Computing the full Jacobian of a composition of functions can be done by *forward-mode* or *reverse-mode* differentiation. Explain the difference, and state which one is used for training a neural network, and why.
- c) Suppose $\mathbf{o} = \mathbf{f}_4(\mathbf{f}_3(\mathbf{f}_2(\mathbf{f}_1(\mathbf{x}))))$. Write down the chain-rule formula for the Jacobian $\partial \mathbf{o} / \partial \mathbf{x}$ in terms of the Jacobians of $\mathbf{f}_1, \dots, \mathbf{f}_4$, making clear the order in which they are multiplied.

- d) For the cross-entropy loss layer, $z = -\sum_c y_c \log(\text{softmax}(\mathbf{x})_c)$, the Jacobian with respect to the logits $\mathbf{x}$ has a remarkably simple closed form. Write it down, using $\mathbf{p} = \text{softmax}(\mathbf{x})$.

Answer:

- a) Backpropagation computes the gradient of the loss with respect to every parameter (and, incidentally, the input), by applying the chain rule layer by layer. This gradient tells us how to update the parameters during gradient-based optimization.
- b) Forward mode builds $\mathbf{J}\mathbf{v}$ by composing Jacobians starting from the input side and working outward; it is cheap when the number of inputs n is smaller than the number of outputs m . Reverse mode builds $\mathbf{u}^\top \mathbf{J}$ starting from the output side and working inward; it is cheap when $n > m$. Neural network training has a scalar loss as output ($m = 1$) but many parameters (n large), so reverse mode is far cheaper – this is exactly backpropagation.
- c) $\frac{\partial \mathbf{o}}{\partial \mathbf{x}} = \mathbf{J}_{f_4}(\mathbf{x}_4) \mathbf{J}_{f_3}(\mathbf{x}_3) \mathbf{J}_{f_2}(\mathbf{x}_2) \mathbf{J}_{f_1}(\mathbf{x})$ – the Jacobians are multiplied in reverse order of composition, with the outermost function’s Jacobian on the left.
- d) $\mathbf{J} = \frac{\partial z}{\partial \mathbf{x}} = (\mathbf{p} - \mathbf{y})^\top$ – the predicted probabilities minus the true label.

Question 5

In a deep network, the gradient of the loss with respect to the hidden units at layer l can be written $\mathbf{g}_l = \mathbf{J}_l \mathbf{g}_{l+1}$, where $\mathbf{J}_l$ is a Jacobian.

- a) Explain informally why, after passing through many layers, this gradient can either vanish or explode, in relation to the eigenvalues of $\mathbf{J}$.
- b) Name one technique that fixes exploding gradients, and explain how it works.
- c) Name two complementary techniques used to address the (harder) problem of vanishing gradients.

Answer:

- a) Backpropagation involves multiplying the gradient by many Jacobian matrices. If $\mathbf{x}_{l+1} = \mathbf{f}_l(\mathbf{x}_l)$, then the Jacobian from layer l to layer L is $\frac{\partial \mathbf{x}_L}{\partial \mathbf{x}_l} = \mathbf{J}_{L-1} \mathbf{J}_{L-2} \cdots \mathbf{J}_l$, where $\mathbf{J}_k = \frac{\partial \mathbf{x}_{k+1}}{\partial \mathbf{x}_k}$. Thus the gradient is repeatedly multiplied by layer-wise Jacobians. If these Jacobians tend to stretch vectors, the gradient can grow rapidly; if they tend to shrink vectors, the gradient can decay toward zero. As a simplified intuition, suppose all layer Jacobians are equal: $\mathbf{J}_k = \mathbf{J}$ for all k . Then $\mathbf{J}_{L-1} \mathbf{J}_{L-2} \cdots \mathbf{J}_l = \mathbf{J}^{L-l}$. Repeated multiplication by $\mathbf{J}$ behaves like repeated multiplication by its eigenvalues. If the spectral radius satisfies $\rho(\mathbf{J}) > 1$, components in expanding directions grow exponentially, leading to exploding gradients. If $\rho(\mathbf{J}) < 1$ components shrink exponentially, leading to vanishing gradients. Therefore, gradients can **vanish** or **explode** because backpropagation multiplies many Jacobians across layers.
- b) **Gradient clipping**: rescale the gradient vector $\mathbf{g}$ to $\mathbf{g}' = \min(1, c/\|\mathbf{g}\|) \mathbf{g}$, capping its norm at c without changing its direction.
- c) Any two of: non-saturating activation functions (ReLU and its variants); additive/residual connections; standardizing activations at each layer; careful parameter initialization.

Question 6

Good initial values for the weights matter a lot when the training objective is non-convex.

- a) Consider a single linear unit $o_i = \sum_{j=1}^{n_{\text{in}}} w_{ij} x_j$, with $w_{ij} \sim \mathcal{N}(0, \sigma^2)$ and inputs with $\mathbb{E}[x_j] = 0$, $\mathbb{V}[x_j] = \gamma^2$, independent of the weights. Write down the formula for $\mathbb{V}[o_i]$ in terms of n_{in} (the fan-in), σ^2 , and γ^2 , and explain why we want $n_{\text{in}} \sigma^2 \approx 1$.
- b) Write down the Xavier/Glorot formula for σ^2 , and explain what it tries to balance.
- c) Which initialization rule is typically paired with ReLU, and which with SELU?

Answer:

- a) $\mathbb{V}[o_i] = n_{\text{in}} \sigma^2 \gamma^2$. We want $n_{\text{in}} \sigma^2 \approx 1$ so that the variance of the activations stays roughly stable as it passes through many layers, rather than shrinking toward 0 or blowing up, which would cause saturated/dead units or unstable gradients.
- b) $\sigma^2 = \frac{2}{n_{\text{in}} + n_{\text{out}}}$. It tries to balance the forward pass (which depends on the fan-in) and the backward pass (which depends on the fan-out), since satisfying $n_{\text{in}} \sigma^2 = 1$ and $n_{\text{out}} \sigma^2 = 1$ exactly at the same time is not generally possible.
- c) **He initialization** ($\sigma^2 = 2/n_{\text{in}}$) for ReLU and its variants; **LeCun initialization** ($\sigma^2 = 1/n_{\text{in}}$) for SELU.

Question 7

Large DNNs can easily overfit, and they can also be slow to train.

- a) Name three different regularization strategies used to control overfitting in deep networks, and briefly describe one of them.
- b) What is the difference between data parallelism and model parallelism when training across multiple machines?
- c) What is the difference between synchronous and asynchronous (“Hogwild”) data-parallel training?

Answer:

- a) Any three of: **early stopping** (stop training once validation error starts to rise); **weight decay** (a Gaussian prior on the weights, shrinking them toward 0, equivalent to an ℓ_2 penalty); **sparse DNNs** (ℓ_1 regularization, driving many weights to exactly zero); **dropout** (randomly zeroing a unit’s outgoing connections with probability p during training); **Bayesian neural networks** (averaging predictions over a posterior distribution of parameters instead of a single point estimate).
- b) **Data parallelism**: every machine keeps a full copy of the model but processes a different slice of the data, and gradients are combined via an all-reduce sum. **Model parallelism**: the model itself is split across machines, which requires tight communication between them.
- c) **Synchronous**: every machine waits for the combined gradient before updating – equivalent to training on one machine with a bigger batch, just faster. **Asynchronous (Hogwild)**: machines update immediately using only their own local gradient – faster per step, but not guaranteed to behave like standard stochastic gradient descent.